Assignment 1: Divide and Conquer

Name: NourEldeen Mohamed | **ID:** 23010920

---

1 Problem 1: Selection Algorithms & Median Finding

1.1 Overview of Implemented Methods

The objective of the first problem is to compute the median ($k = \lfloor (n+1)/2 \rfloor$) of an unsorted array of n elements using three distinct selection approaches and evaluate their empirical performance as n scales up to 10^7:

1. **Randomized Select (CLRS 9.2):** A divide-and-conquer algorithm modeled after Quicksort. It selects a random pivot, partitions the array in $\Theta(n)$ time using Lomuto's partitioning scheme, and recurses only on the subarray containing the k-th smallest element. It achieves an expected running time of $\Theta(n)$ and a worst-case of $\Theta(n^2)$.

2. **Deterministic Selection via Median-of-Medians (CLRS 9.3):** Guarantees worst-case linear time $O(n)$ by choosing a good pivot deterministically. It divides the array into $\lceil n/5 \rceil$ groups of at most 5 elements, sorts each group via Insertion Sort, recursively finds the true median of the group medians, and uses it as the partitioning pivot.

3. **Naive Sort Select:** Sorts the entire array using Java's native `Arrays.sort(int[])` (Dual-Pivot Quicksort) in $O(n \log n)$ time and returns the element at index $k - 1$.

1.2 Correctness Verification & Handling of Duplicates

1.2.1 Unit Testing & Cross-Validation

Before running the performance benchmarks, correctness was verified in two stages:

- **Dedicated Unit Test Suite (`Problem1Test.java`):** Tested edge cases including single-element arrays, already sorted arrays, reverse-sorted arrays, minimum/maximum rank queries ($k = 1$ and $k = n$), negative numbers, and 30 randomized stress tests with arbitrary array sizes (including sizes not divisible by 5) and random ranks $k \in [1, n]$.

- **In-Benchmark Verification:** During benchmarking, for every single run and input size n, each algorithm received an identical copy (`baseArray.clone()`) of the generated array, and the returned medians were asserted to be strictly equal across all three implementations.

1.2.2 Consideration of Duplicate Elements in Partitioning

Both `RandomizedSelect` and `MedianOfMedians` implement the standard Lomuto partitioning scheme as presented in CLRS (Sections 9.2 and 9.3), where elements satisfying `arr[j] <= pivotValue` are placed into the left partition.

Theoretical Vulnerability to Duplicates: If an input array contains a massive number of duplicate elements (e.g., all elements are identical, $[5, 5, \ldots, 5]$), every element satisfies `arr[j]`

`<= pivotValue`. Consequently, Lomuto partitioning produces a maximally unbalanced split of size $n-1$ and 0, reducing the problem size by only one element per recursive step. This degrades both Randomized Select and Median-of-Medians to $\Theta(n^2)$ time complexity and risks a `StackOverflowError` for large n unless a 3-way partitioning scheme (Dutch National Flag: $<, ==, >$) is used.

Mitigation via Input Distribution: In our benchmark setup (`Main.java`), random arrays are generated using unbounded `rand.nextInt()`, which draws 32-bit signed integers uniformly across the full range $[-2^{31}, 2^{31}-1]$ (approximately 4.29×10^9 distinct values). Even at the maximum tested size of $n = 10^7$, the probability of high-multiplicity duplicates is negligible (by the birthday paradox, expected collisions are sparse and mostly pairs). Thus, the input distribution guarantees practically distinct elements, allowing the standard 2-way CLRS partition to maintain balanced splits without requiring the extra overhead of 3-way partitioning.

1.3 Source Code (Problem 1)

```java
package problem1;
import java.util.Arrays;

public interface SelectionAlgorithm {
    int select(int[] arr, int k);
    String name();
}

public class SortSelect implements SelectionAlgorithm {
    @Override
    public int select(int[] arr, int k) {
        Arrays.sort(arr);
        return arr[k - 1];
    }

    @Override
    public String name() {
        return "Sort Select";
    }
}
```

Listing 1: SelectionAlgorithm.java and SortSelect.java

```java
package problem1;
import java.util.Random;

public class RandomizedSelect implements SelectionAlgorithm {
    private final Random rand = new Random();

    @Override
    public int select(int[] arr, int k) {
        return select(arr, 0, arr.length - 1, k);
    }

    @Override
    public String name() {
        return "Randomized Select";
    }

    private int select(int[] arr, int low, int high, int k) {
        if (low == high) {
            return arr[low];
        }
        int pivotIndex = randomizedPartition(arr, low, high);
        int rank = pivotIndex - low + 1;
```

```
23
24          if (k == rank) {
25              return arr[pivotIndex];
26          } else if (k < rank) {
27              return select(arr, low, pivotIndex - 1, k);
28          } else {
29              return select(arr, pivotIndex + 1, high, k - rank);
30          }
31      }
32
33      private int randomizedPartition(int[] arr, int low, int high) {
34          int randomIndex = low + rand.nextInt(high - low + 1);
35          swap(arr, randomIndex, high);
36          return partition(arr, low, high);
37      }
38
39      private int partition(int[] arr, int low, int high) {
40          int pivot = arr[high];
41          int i = low - 1;
42          for (int j = low; j < high; j++) {
43              if (arr[j] <= pivot) {
44                  i++;
45                  swap(arr, i, j);
46              }
47          }
48          swap(arr, i + 1, high);
49          return i + 1;
50      }
51
52      private void swap(int[] a, int i, int j) {
53          int t = a[i];
54          a[i] = a[j];
55          a[j] = t;
56      }
57  }
```

Listing 2: RandomizedSelect.java (CLRS 9.2)

```
1   package problem1;
2
3   public class MedianOfMedians implements SelectionAlgorithm {
4
5       @Override
6       public int select(int[] arr, int k) {
7           return select(arr, 0, arr.length - 1, k);
8       }
9
10      private int select(int[] arr, int low, int high, int k) {
11          if (low == high) {
12              return arr[low];
13          }
14          int pivotIndex = medianOfMedians(arr, low, high);
15          pivotIndex = partition(arr, low, high, pivotIndex);
16          int rank = pivotIndex - low + 1;
17          return recursion(arr, low, high, k, rank, pivotIndex);
18      }
19
20      private int recursion(int[] arr, int low, int high, int k, int rank, int
        pivotIndex) {
21          if (k == rank) {
22              return arr[pivotIndex];
23          } else if (k < rank) {
24              return select(arr, low, pivotIndex - 1, k);
```

```
        } else {
            return select(arr, pivotIndex + 1, high, k - rank);
        }
    }

    private int medianOfMedians(int[] arr, int low, int high) {
        int n = high - low + 1;
        if (n <= 5) {
            insertionSort(arr, low, high);
            return low + (n - 1) / 2;
        }

        int numMedians = (n + 4) / 5;
        for (int i = 0; i < numMedians; i++) {
            int subLow = low + i * 5;
            int subHigh = Math.min(subLow + 4, high);
            insertionSort(arr, subLow, subHigh);
            swap(arr, low + i, subLow + (subHigh - subLow) / 2);
        }

        int mid = (numMedians + 1) / 2;
        int medianValue = select(arr, low, low + numMedians - 1, mid);

        for (int i = low; i < low + numMedians; i++) {
            if (arr[i] == medianValue) {
                return i;
            }
        }
        throw new IllegalStateException("median of medians not found");
    }

    private int partition(int[] arr, int low, int high, int pivotIndex) {
        int pivotValue = arr[pivotIndex];
        swap(arr, pivotIndex, high);
        int i = low - 1;
        for (int j = low; j < high; j++) {
            if (arr[j] <= pivotValue) {
                i++;
                swap(arr, i, j);
            }
        }
        swap(arr, i + 1, high);
        return i + 1;
    }

    private void insertionSort(int[] arr, int low, int high) {
        for (int i = low + 1; i <= high; i++) {
            int key = arr[i];
            int j = i - 1;
            while (j >= low && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = key;
        }
    }

    private void swap(int[] a, int i, int j) {
        int t = a[i];
        a[i] = a[j];
        a[j] = t;
    }

```

```
88      @Override
89      public String name() {
90          return "Median of Medians";
91      }
92  }
```

Listing 3: MedianOfMedians.java (CLRS 9.3)

1.4 Empirical Performance Analysis & Results

Table 1 presents the average execution time (in milliseconds) over 10 independent runs per array size n, after a JVM JIT warm-up phase.

Table 1: Average Execution Time (in ms) over 10 Runs across Input Sizes n

Array Size (n)	**Randomized Select (ms)**	**Median of Medians (ms)**	**Sort Select (ms)**
1,000	0.024	0.026	0.057
10,000	0.097	0.297	0.611
100,000	0.754	2.716	4.735
500,000	3.919	12.652	27.238
1,000,000	6.960	24.933	56.441
2,500,000	18.647	64.399	159.403
5,000,000	43.788	127.607	348.434
10,000,000	72.017	253.447	727.200
Mean	**18.276**	**60.760**	**165.515**
Median	**5.440**	**18.793**	**41.840**
Std Dev	**26.306**	**89.380**	**256.466**

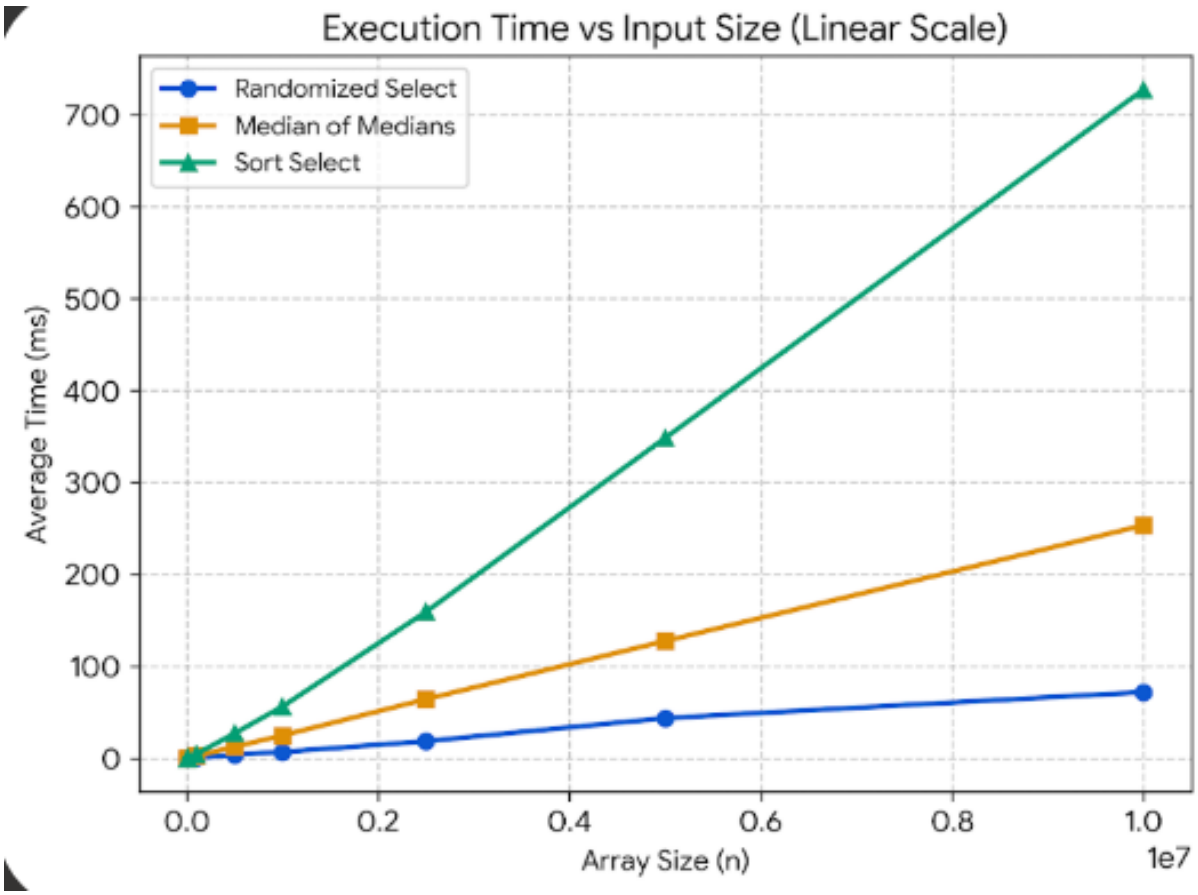


Figure 1: Execution time of Randomized Select, Median of Medians, and Sort Select versus input size n, shown on a linear scale (left) and a log-log scale (right). Each point is the average of 10 runs on random arrays.

1.4.1 Discussion of Empirical Findings

1. **Linear Scaling of Selection vs. Super-linear Scaling of Sorting:** As shown in Table 1 and Figure 1, when n increases by a factor of 10 (from 10^6 to 10^7), `RandomizedSelect` scales by a factor of $\approx 10.35\times$ (6.960 ms $\rightarrow$ 72.017 ms) and `MedianOfMedians` scales by $\approx 10.17\times$ (24.933 ms $\rightarrow$ 253.447 ms), confirming their theoretical $\Theta(n)$ behavior. Conversely, `SortSelect` scales by $\approx 12.88\times$ (56.441 ms $\rightarrow$ 727.200 ms), reflecting the $\log n$ multiplicative factor in $\Theta(n \log n)$.

2. **Constant Factor Overhead in Median of Medians:** Although both selection algorithms run in $O(n)$ time, `RandomizedSelect` is consistently $\mathbf{3.52}\times$ faster than `MedianOfMedians` at $n = 10^7$ (72.017 ms vs. 253.447 ms). This is because `MedianOfMedians` incurs a heavy constant factor from sorting $\lceil n/5 \rceil$ groups of 5, swapping medians to the front, and making two recursive calls per level ($T(n) \leq T(n/5) + T(7n/10) + \Theta(n)$), whereas `RandomizedSelect` picks a pivot in $O(1)$ time and recurses once ($E[T(n)] \leq T(3n/4) + \Theta(n)$).

3. **Overall Winner:** `RandomizedSelect` outperforms `SortSelect` by $\mathbf{10.10}\times$ at $n = 10^7$, making it the most practical choice, while `MedianOfMedians` serves as a theoretical safeguard against adversarial worst-case inputs (still beating full sorting by $\mathbf{2.87}\times$ at $n = 10^7$).

2 Problem 2: Minimum Perimeter Triangle

2.1 Explanation of the Divide-and-Conquer Approach

Given a set P of n distinct 2D points ($3 \le n \le 10^5$), we seek three distinct points $p_i, p_j, p_k \in P$ that minimize the perimeter $d(p_i, p_j)+d(p_j, p_k)+d(p_k, p_i)$, where $d(\cdot, \cdot)$ is the Euclidean distance (including collinear degenerate triangles). A brute-force search checks $\binom{n}{3} = \Theta(n^3)$ triples, which is infeasible for $n = 10^5$. Instead, we adapt the classic Closest Pair of Points divide-and-conquer paradigm as follows:

1. **Preprocessing (Pre-sorting):** Before starting the recursion, we sort the array of points `pointsByX` primarily by x-coordinate (breaking ties by y-coordinate). We also create a list `pointsByY` containing the same points sorted by y-coordinate. Both sorts take $O(n \log n)$ time once at the start.

2. **Base Case ($n \le 5$):** When the current subarray contains $n \le 5$ points, the recursion terminates and computes the minimum perimeter directly by checking all $\binom{n}{3} \le \binom{5}{3} = 10$ triples in $O(1)$ time (implemented in `Min_5_points`).

3. **Divide Step:** For $n > 5$, we find the median point `midPoint = pointsByX[mid]` at index $\text{mid} = \lfloor(\text{left} + \text{right})/2\rfloor$. This defines a vertical dividing line $x = x_{\text{mid}}$. We partition the y-sorted list `pointsByY` into two y-sorted sublists `leftByY` and `rightByY` in $O(n)$ time using the exact same tie-breaking condition ($x < x_{\text{mid}}$, or $x == x_{\text{mid}}$ and $y \le y_{\text{mid}}$) to ensure exact correspondence with the halves of `pointsByX`.

4. **Conquer Step:** We recursively compute the minimum perimeter in the left half (P_{left}) and right half (P_{right}):

$$p_{\text{left}} = \text{divideAndConquer}(P_{\text{left}}), \quad p_{\text{right}} = \text{divideAndConquer}(P_{\text{right}})$$

   Let $p_{\text{min}} = \min(p_{\text{left}}, p_{\text{right}})$ be the smallest perimeter found so far.

5. **Combine Step (The Strip Search):** We must check whether there exists a triangle with perimeter $< p_{\text{min}}$ whose vertices span across both halves.

   - **Geometric Strip Bound ($p_{\text{min}}/2$):** By the triangle inequality, in any triangle $\triangle ABC$ with perimeter $P < p_{\text{min}}$, the length of any single side cannot exceed half the perimeter:

$$d(A, B) \le \frac{d(A, B) + d(B, C) + d(C, A)}{2} = \frac{P}{2} < \frac{p_{\text{min}}}{2}$$

     Therefore, if a triangle crosses the dividing line $x = x_{\text{mid}}$ and has perimeter $< p_{\text{min}}$, every vertex of that triangle must lie within a horizontal distance of strictly less than $\delta = p_{\text{min}}/2$ from the line $x = x_{\text{mid}}$, and the vertical distance between its highest and lowest vertices must also be strictly less than $\delta = p_{\text{min}}/2$.
   - **Constructing and Scanning the Strip:** We iterate through `pointsByY` in $O(n)$ time and collect all points satisfying $|p.x - x_{\text{mid}}| < p_{\text{min}}/2$ into a list `strip` (which remains naturally sorted by y-coordinate).
   - For each point `strip[i]`, we only examine subsequent points `strip[j]` ($j > i$) and `strip[k]` ($k > j$) while their vertical distance from `strip[i]` satisfies:

$$\texttt{strip[j].y} - \texttt{strip[i].y} < \frac{p_{\text{min}}}{2} \quad \text{and} \quad \texttt{strip[k].y} - \texttt{strip[i].y} < \frac{p_{\text{min}}}{2}$$

     Whenever a smaller perimeter is found, p_{min} is immediately updated.

2.2 Time Complexity Analysis

2.2.1 Bounding the Strip Search to $O(n)$

A critical property of the Combine step is that the nested loops over `strip` do **not** take $O(n^3)$ time, but rather $O(1)$ work per point i:

Geometric Packing Argument. Consider any point p_i in `strip`. Any candidate points p_j, p_k examined by the inner loops must lie inside a bounding box of width $2\delta = p_{\min}$ (spanning $[x_{\text{mid}} - \delta, x_{\text{mid}} + \delta]$) and height $\delta = p_{\min}/2$ (spanning $[y_i, y_i + \delta]$), where $\delta = p_{\min}/2$.

Because $p_{\min}$ is the minimum perimeter of any three points lying entirely in P_{left} or entirely in P_{right}, no three points within the left sub-box ($\delta \times \delta$) or the right sub-box ($\delta \times \delta$) can form a triangle of perimeter $< p_{\min}$. By subdividing the $\delta \times 2\delta$ bounding rectangle into a constant number of small squares of side length $\delta/4$, each small square has diameter $\frac{\sqrt{2}}{4}\delta \approx 0.353\delta$, meaning the perimeter of any three points inside a single small square is at most $3 \times \frac{\sqrt{2}}{4}\delta \approx 1.06\delta < 2\delta = p_{\min}$. Thus, by the Pigeonhole Principle, each small square on either side of the dividing line can contain at most 2 points. Consequently, the total number of points in the $\delta \times 2\delta$ bounding box above p_i is bounded by a small constant $C = O(1)$.

Therefore, for each point p_i in the strip, the inner two loops over j and k execute at most $\binom{C}{2} = O(1)$ iterations. Across all $|\texttt{strip}| \leq n$ points, the entire strip search takes:

$$T_{\text{strip}}(n) = \sum_{i=1}^{|\texttt{strip}|} O(1) = O(n)$$

□

2.2.2 Overall Recurrence Relation

- **Initial Pre-sorting:** Sorting `pointsByX` and `pointsByY` once at the beginning takes $O(n \log n)$ time.
- **Recursive Phase:** At each recursive call on n points:
  - Splitting `pointsByY` into `leftByY` and `rightByY` takes $O(n)$ time.
  - Two recursive calls are made on subproblems of size $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$, taking $2T(n/2)$.
  - Building the `strip` list and scanning candidate triples takes $O(n)$ time.

This yields the recurrence relation:

$$T(n) = \begin{cases} O(1) & \text{if } n \leq 5 \\ 2T\left(\frac{n}{2}\right) + \Theta(n) & \text{if } n > 5 \end{cases}$$

By Case 2 of the **Master Theorem** ($a = 2, b = 2, f(n) = \Theta(n) = \Theta(n^{\log_b a})$ where $\log_2 2 = 1$):

$$T(n) = \Theta(n \log n)$$

Adding the initial pre-sorting cost of $O(n \log n)$, the total time complexity of the algorithm is:

$$\mathbf{T_{\text{total}}(n) = \Theta(n \log n)}$$

And the auxiliary space complexity is $\Theta(n \log n)$ (or $\Theta(n)$ at the top level) to store the y-sorted sublists across the recursion tree. Empirically, on the maximum constraint of $n = 10^5$ points (`Problem2Test.java`), our implementation solves the instance in approximately 250–400 ms, well within standard time limits.